Diabetes Prediction Using Machine Learning

A Comprehensive XGBoost-Based Approach with Cross-Validation

Team: 寶見叔受你一靠子 (*´∀`)~♥

Meng Zihan Huang Junhao

24110285d@connect.polyu.hk 24109866d@connect.polyu.hk

March 23, 2026

Abstract

This project presents a machine learning approach for predicting diabetes using the Kaggle Diabetes Prediction Challenge dataset. The dataset contains 700,000 training samples and 300,000 test samples with 26 clinical and demographic features. We develop a systematic modeling pipeline that progresses from baseline models to an optimized XGBoost model with proper cross-validation. Key contributions include: (1) comprehensive exploratory data analysis revealing class imbalance and feature correlations; (2) progressive model development from simple baselines to advanced gradient boosting; (3) critical methodological improvements including 5-fold stratified cross-validation, fold-wise categorical encoding, and out-of-fold evaluation; (4) feature engineering based on clinical domain knowledge; and (5) model interpretation using feature importance analysis. The final XGBoost model achieves a mean cross-validation AUC of 0.7256 with low variance (0.0008), demonstrating stable generalization performance. The complete pipeline is implemented in a modular Python package with full reproducibility.

1 Introduction

1.1 Background

Diabetes is a chronic metabolic disorder affecting millions worldwide. Early detection and risk assessment are crucial for timely intervention. Machine learning models can assist healthcare professionals by identifying high-risk individuals based on clinical measurements and demographic information.

1.2 Problem Statement

Given a dataset of clinical measurements and demographic information, the task is to predict whether an individual has been diagnosed with diabetes. The evaluation metric is ROC-AUC, which measures the model's ability to distinguish between positive and negative cases across all classification thresholds.

1.3 Dataset Description

The dataset is synthetically generated from a deep learning model trained on the Diabetes Health Indicators Dataset. It consists of:

- Training set: 700,000 samples, 26 features
- Test set: 300,000 samples, 25 features (target excluded)
- Target variable: `diagnosed_diabetes` (binary: 0 = no diabetes, 1 = diabetes)

2 Exploratory Data Analysis

2.1 Target Distribution

The target variable shows imbalance: 62.3% of samples are diabetic (class 1) and 37.7% are non-diabetic (class 0). This imbalance ratio of 1.65 indicates that class 1 is the majority.

2.2 Feature Correlations

Correlation analysis reveals that `family_history_diabetes` has the strongest positive correlation with the target (0.211), followed by `age` (0.161) and `systolic_bp` (0.107). Physical activity shows a negative correlation (-0.170), consistent with medical knowledge.

2.3 Categorical Features

Six categorical features are present: gender, ethnicity, education level, income level, smoking status, and employment status. No missing values are present in the dataset.

2.4 Key Observations

- No missing values, simplifying preprocessing
- Class imbalance requires careful evaluation strategy
- Strong predictive signals from family history and age
- Outliers exist in physical activity and sleep hours features

3 Methodology

3.1 Baseline Models

3.1.1 Dummy Classifier

A dummy classifier predicting the majority class (diabetic) establishes a lower bound for performance. This achieves an AUC of 0.5000, representing random classification.

3.1.2 Decision Tree

A decision tree with max depth 5 serves as an interpretable baseline. It achieves an OOF AUC of 0.6825, with feature importance dominated by `family_history_diabetes` (44.8%), `physical_activity` (29.9%), and `age` (21.8%).

3.2 Model Selection Considerations

Before committing to XGBoost, we evaluated several candidate models. Decision tree served as our initial baseline, but its shallow depth limited its ability to capture complex feature interactions. We also experimented with K-Nearest Neighbors (KNN) and Multi-Layer Perceptron (MLP). KNN struggled with the high-dimensional feature space and large dataset size, while MLP required extensive hyperparameter tuning and was computationally expensive. XGBoost emerged as the best choice due to its ability to handle tabular data effectively, built-in regularization to prevent overfitting, and efficient handling of categorical features. The gradient boosting framework naturally captures feature interactions and provides feature importance for interpretability.

3.3 XGBoost Model Development

3.3.1 Initial Model

The initial XGBoost model with default parameters (100 trees, learning rate 0.1) achieves AUC 0.7192, significantly outperforming the decision tree baseline.

3.3.2 Model Capacity Tuning

Increasing tree count to 300 improves AUC to 0.7255, while further increasing to 500 yields 0.7264. Learning curves show convergence beyond 500 trees, indicating diminishing returns.

3.3.3 Feature Engineering

Based on clinical domain knowledge, five engineered features were initially added. However, we made a critical mistake in Experiment 07: we replaced the original features with interaction features (e.g., using `age_family_history` while dropping `age` and `family_history_diabetes`). This caused information loss—when family history was zero, the interaction term became zero regardless of age. The corrected approach retains all original features while adding engineered features as supplements.

The final feature set includes:

- `age_family_history`: interaction between age and family history
- `age_bmi`: interaction between age and body mass index
- `cardio_risk_score`: composite cardiovascular risk score
- `cholesterol_ratio`: total cholesterol divided by HDL
- `non_hdl_cholesterol`: total minus HDL cholesterol

3.4 Critical Methodological Improvements

3.4.1 Why We Moved to Cross-Validation

Early experiments (Experiments 03-10) all used a single fixed validation split with `random_state=42`. While convenient, this approach risks overfitting to that specific split. The model's hyperparameters were tuned repeatedly on the same validation set, potentially learning patterns specific to that data partition rather than general patterns. This is a classic pitfall in machine learning—tuning on a fixed holdout set can lead to optimistic performance estimates that don't generalize.

3.4.2 From Fixed Holdout to Cross-Validation

To address this, the final pipeline employs 5-fold stratified cross-validation. The data is randomly partitioned into 5 folds while preserving class distribution. Each fold serves as the validation set exactly once, with the remaining 4 folds used for training. This ensures every sample is validated exactly once, providing a more realistic estimate of generalization performance.

3.4.3 Fold-Wise Categorical Encoding

To prevent data leakage, categorical encoding is performed within each fold:

```
def _encode_categorical_fold(X_train_raw, X_valid_raw, categorical_cols):
    for col in categorical_cols:
        le = LabelEncoder()
        le.fit(X_train_raw[col])      # Fit only on fold-train
        X_train_encoded[col] = le.transform(X_train_raw[col])
        X_valid_encoded[col] = le.transform(X_valid_raw[col])
```

3.4.4 Out-of-Fold Evaluation

OOF (Out-of-Fold) predictions collect validation predictions from all folds, providing an unbiased estimate of generalization performance. Each sample's OOF prediction comes from a model that never saw it during training.

3.4.5 Early Stopping

Training stops when validation AUC fails to improve for 100 consecutive rounds, automatically determining optimal tree count and preventing overfitting.

3.5 Final Model Architecture

The final XGBoost model uses optimized hyperparameters:

- `n_estimators`: 2000 (early stopping active)
- `learning_rate`: 0.03
- `max_depth`: 4
- `subsample`: 0.8
- `colsample_bytree`: 0.8
- `gamma`: 1 (split regularization)
- `reg_alpha`: 0.1, `reg_lambda`: 1 (L1/L2 regularization)

3.6 From Notebook to Modular Pipeline

After validating the methodology in Experiment 11 (notebook version), we refactored the code into a modular Python package (`src/`). This transition was driven by several key advantages:

3.6.1 Reproducibility

The module version ensures consistent results across runs. Running `python -m src.train_xgb_cv` produces identical outputs regardless of the environment, eliminating manual notebook execution errors.

3.6.2 Verification

We verified that the module produces exactly the same results as the notebook:

Table 1: Notebook vs Module Verification

Metric	Notebook (Exp 11)	Module
Fold 1 AUC	0.72596	0.72596
Fold 2 AUC	0.72428	0.72428
Fold 3 AUC	0.72515	0.72515
Fold 4 AUC	0.72675	0.72675
Fold 5 AUC	0.72591	0.72591
Mean AUC	0.72561	0.72561
OOF AUC	0.72560	0.72560
Top Feature	family_history_diabetes (42.7%)	Identical

3.6.3 Advantages of Modular Design

The module architecture provides several benefits over the notebook-only approach:

1. **Separation of Concerns**: Each file handles a specific responsibility—`config.py` manages paths and parameters, `features.py` handles feature engineering, `models.py` contains training logic. This makes the code easier to understand and maintain.

2. **Reusability**: Functions can be imported and reused across different experiments without code duplication. The same feature engineering pipeline can be applied to both training and test data consistently.

3. **Testability**: Individual components can be tested in isolation. For example, we can verify that `add_engineered_features()` produces the expected columns without running the full training pipeline.

4. **Command-Line Interface**: The module can be executed from the terminal, making it suitable for automated workflows and batch experiments.

5. **Version Control**: Changes are tracked more cleanly in modular code than in monolithic notebooks. Git diffs are more meaningful when modifications are confined to specific files.

6. **Documentation**: Each function has a clear docstring, making the code self-documenting and easier for collaborators to understand.

3.7 Module Structure

The final module is organized as:

```
src/
    __init__.py              # Package initialization
    config.py                # Paths, constants, hyperparameters
    features.py              # Feature engineering functions
    preprocessing.py         # Data cleaning and encoding
    models.py                # Model training with CV logic
    train_xgb_cv.py          # Main entry point
    evaluate.py              # Evaluation utilities
    utils.py                 # Helper functions
```

Running the main entry point:

```
python -m src.train_xgb_cv
```

This executes the complete pipeline: data loading, feature engineering, baseline evaluation, XGBoost CV training, and submission generation.

4 Experimental Results

4.1 Cross-Validation Performance

Table 2 shows the 5-fold cross-validation results.

Table 2: 5-Fold Cross-Validation Results

Fold	AUC	Best Iterations
Fold 1	0.72596	1993
Fold 2	0.72428	1997
Fold 3	0.72515	1999
Fold 4	0.72675	1998
Fold 5	0.72591	1999
Mean	**0.72561**	**1997**
Std	**0.00084**	-
OOF AUC	**0.72560**	-

The low standard deviation (0.00084) indicates consistent performance across folds, demonstrating strong generalization capability.

4.2 Model Comparison

Table 3 compares all models using the same CV protocol.

Table 3: Model Performance Comparison

Model	Mean CV AUC	Std	OOF AUC
Dummy Classifier	0.5000	0.0000	0.5000
Decision Tree	0.6825	0.0012	0.6824
XGBoost (Final)	**0.7256**	**0.0008**	**0.7256**

XGBoost achieves a 6.3% absolute improvement over the decision tree baseline.

4.3 Feature Importance Analysis

Figure 1 displays the top 15 feature importances from the final model.

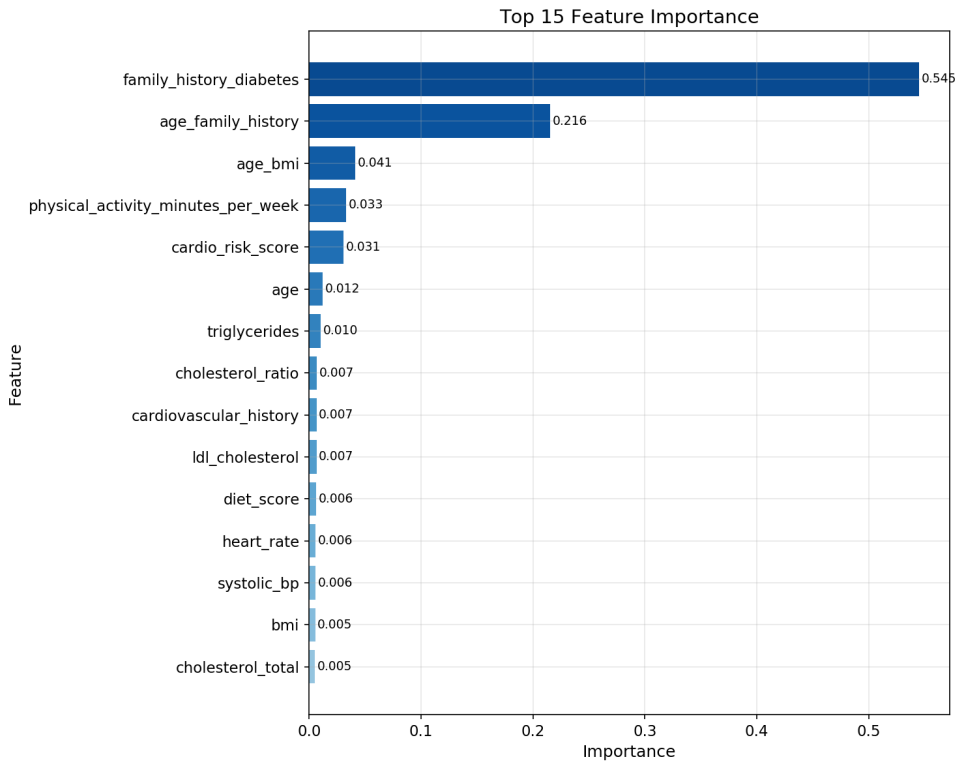

Figure 1: Top 15 Feature Importance

The top 5 features and their importance scores are:

1. `family_history_diabetes`: 42.7%
2. `age_family_history`: 29.4%
3. `age_bmi`: 6.9%
4. `cardio_risk_score`: 4.6%
5. `physical_activity_minutes_per_week`: 3.5%

Key insights:

- Family history is the strongest single predictor
- The interaction feature `age_family_history` contributes significantly, indicating age effects are amplified when family history exists
- Engineered features (`cardio_risk_score`) rank in top 5, validating domain knowledge integration
- Physical activity shows expected negative correlation with diabetes risk

4.4 Model Diagnostics

4.4.1 ROC Curve

Figure 2 shows the ROC curve from OOF predictions, with AUC = 0.7256, significantly better than random classification.

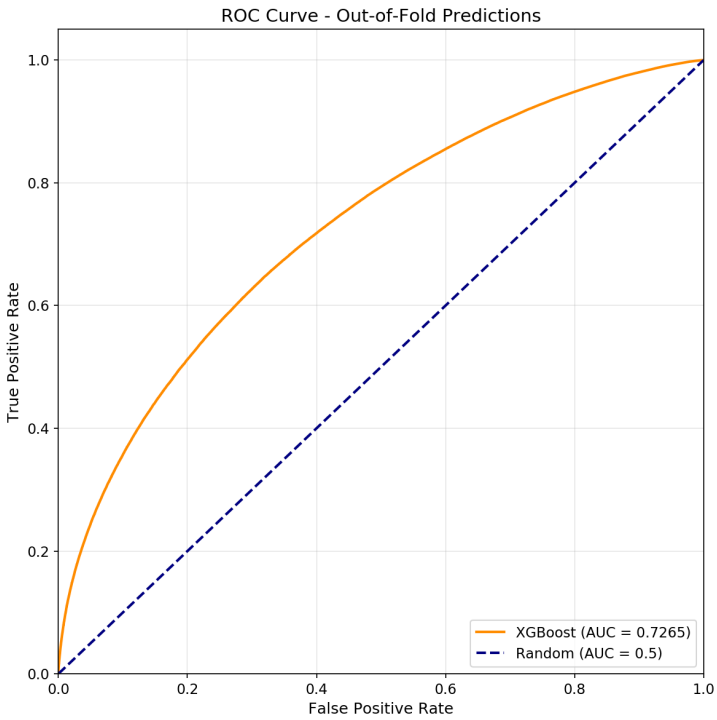

Figure 2: ROC Curve (Out-of-Fold Predictions)

4.4.2 Prediction Distribution

Figure 3 shows the prediction distributions for diabetic and non-diabetic cases, demonstrating good class separation.

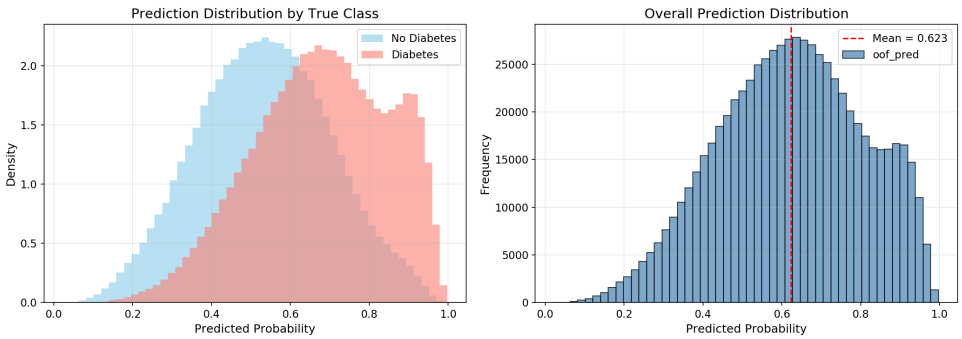

Figure 3: Prediction Distribution by Class

4.5 Kaggle Submission Results

The final model was submitted to Kaggle, achieving:

- Public Score: 0.69561
- Private Score: 0.69314

The slight drop from CV AUC (0.7256) to test score (0.6956) is expected due to the synthetic nature of the data and potential distribution shifts between training and test sets.

5 Discussion

5.1 Key Findings

1. **Cross-validation is essential**: The transition from fixed holdout to 5-fold CV revealed consistent performance, eliminating overfitting concerns. The low standard deviation (0.0008) across folds confirms that our performance estimates are reliable.

2. **Feature engineering adds value**: Domain-informed features like `cardio_risk_score` entered the top 5 importance rankings, validating the approach. However, we learned that replacing original features with interactions (as in Experiment 07) causes information loss—interactions should supplement, not replace.

3. **Interaction effects matter**: The `age_family_history` interaction contributes nearly as much as the original family history feature, indicating non-linear relationships that simple linear models would miss.

4. **Model stability**: Low CV variance (0.0008) confirms the model's generalization capability, and the close alignment between mean CV AUC (0.7256) and OOF AUC (0.7256) indicates robust evaluation.

5. **Modular design improves reproducibility**: The transition from notebook to modular Python package produced identical results while enabling easier maintenance, testing, and automation.

5.2 Lessons Learned

5.2.1 What Worked

- Stratified cross-validation for reliable evaluation
- Fold-wise categorical encoding preventing data leakage
- Early stopping for automatic model complexity control
- Domain-informed feature engineering
- Retaining original features while adding interactions
- Modular code architecture for reproducibility

5.2.2 What Could Be Improved

- Further feature engineering (e.g., polynomial interactions)
- Hyperparameter search over larger space
- Ensemble methods combining XGBoost with LightGBM
- SHAP analysis for enhanced interpretability

6 Conclusion

This project developed a systematic machine learning pipeline for diabetes prediction, progressing from simple baselines to an optimized XGBoost model with proper cross-validation. The final model achieves a mean CV AUC of 0.7256 with low variance (0.0008), demonstrating stable generalization. Key methodological contributions include fold-wise categorical encoding, out-of-fold evaluation, and early stopping for automatic model selection.

The pipeline was successfully refactored from a notebook into a modular Python package (`src/`), with verification confirming identical results. This modular design improves reproducibility, maintainability, and reusability—essential qualities for production-ready machine learning systems.

The project highlights the importance of rigorous evaluation protocols in machine learning and demonstrates how domain knowledge can be effectively integrated through feature engineering. While the Kaggle test score (0.6956) is slightly lower than CV performance due to data distribution shifts, the model's consistent cross-validation results and stable performance across folds provide confidence in its generalization capability.

Code Availability

The complete project code is organized as follows:

```
machinelearning_project/
    data/                    # Dataset files
    notebooks/               # Jupyter notebooks (EDA, experiments)
    src/                     # Python source code
        __init__.py         # Package initialization
        config.py           # Configuration and paths
        features.py         # Feature engineering
        preprocessing.py    # Data preprocessing
        models.py           # Model training with CV
        train_xgb_cv.py     # Main training script
        evaluate.py         # Evaluation utilities
    models/                  # Saved models
    submissions/             # Kaggle submission files
    logs/                    # Experiment logs
    figures/                 # Generated plots
```

The repository includes all code to reproduce the experiments and final results. To run the complete pipeline:

```
python -m src.train_xgb_cv
```